

LAPORAN PRAKTIKUM STRUKTUR DATA

Algoritma Sorting

Disusun Oleh :

Aliifah Felda Mufarrihati Salwaa

2511531011

Dosen Pengampu :

Dr. Wahyudi, M.T

Asisten Praktikum :

M Galid Avaro



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Laporan praktikum ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum Struktur Data yang membahas tentang Algoritma Sorting. Melalui laporan ini penulis dapat memahami materi praktikum secara mendalam dan dapat lebih teliti, teratur, serta memiliki kemampuan yang baik dalam penulisan kode program sesuai kaidah akademik. Sehingga laporan ini dapat menjadi sarana belajar, dokumentasi kegiatan, dan referensi praktikum berikutnya.

Penulis menyadari bahwa laporan ini masih memiliki banyak kekurangan, baik dari isi maupun penyajiannya. Oleh karena itu, saran dan kritik sangat penulis harapkan guna untuk menyempurnakan laporan berikutnya.

Padang, 21 Mei 2026

Aliifah Felda Mufarrihati Salwaa

2511531011

DAFTAR ISI

KATA PENGANTAR.....	i
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang Praktikum.....	3
1.2 Tujuan Praktikum	3
1.3 Manfaat Praktikum	3
BAB II PEMBAHASAN	4
2.1. Dasar Teori	4
2.2. Class InsertionSort_2511531011	5
2.3. SelectionSort_2511531011	6
2.4. BubbleSort_2511531011.....	7
2.5. InsertionGUI_2511531011.....	8
2.6. Tugas Pekan 7	15
BAB III KESIMPULAN.....	28
3.1. Kesimpulan.....	28
3.2. Saran	28
DAFTAR PUSTAKA	29

BAB I

PENDAHULUAN

1.1 Latar Belakang Praktikum

Praktikum algoritma sorting dilakukan untuk memahami cara kerja proses pengurutan data dalam pemrograman, karena sorting merupakan salah satu teknik dasar yang sangat penting dalam pengolahan data. Dengan mempelajari algoritma seperti Insertion Sort, Selection Sort, dan Bubble Sort, praktikan dapat mengetahui bagaimana data dapat diurutkan secara sistematis dari nilai terkecil ke terbesar maupun sebaliknya. Selain itu, praktikum ini membantu mahasiswa memahami logika perbandingan, pertukaran data, serta efisiensi setiap algoritma sorting dalam menyelesaikan suatu permasalahan. Melalui implementasi program dan visualisasi langkah-langkah sorting, mahasiswa diharapkan mampu meningkatkan kemampuan analisis algoritma dan pemrograman secara lebih terstruktur.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut:

1. Memahami tentang algoritma sorting dalam pemrograman.
2. Mengetahui kelebihan dan kekurangan algoritma sorting dalam program.
3. Dapat membangun sebuah kode program dengan algoritma sorting.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah sebagai berikut:

1. Mampu mengimplementasikan algoritma sorting dalam pembuatan program agar lebih efektif dan efisien.
2. Memahami penggunaan algoritma sorting dalam program.

BAB II

PEMBAHASAN

2.1. Dasar Teori

Algoritma sorting adalah metode atau langkah-langkah yang digunakan untuk mengurutkan data berdasarkan urutan tertentu, seperti dari kecil ke besar atau dari besar ke kecil. Sorting sangat penting dalam pemrograman karena dapat mempermudah proses pencarian, pengelolaan, dan analisis data. Dengan data yang terurut, proses pengolahan informasi menjadi lebih cepat dan efisien. Beberapa algoritma sorting yang sering digunakan adalah Insertion Sort, Selection Sort, dan Bubble Sort.

Insertion Sort adalah algoritma pengurutan yang bekerja dengan cara mengambil satu data kemudian menyisipkannya ke posisi yang tepat pada bagian array yang sudah terurut. Algoritma ini mirip seperti proses menyusun kartu secara manual di tangan. Data dibandingkan satu per satu dengan elemen sebelumnya hingga ditemukan posisi yang sesuai. Proses ini dilakukan berulang sampai seluruh data menjadi terurut.

Selection Sort adalah algoritma sorting yang bekerja dengan mencari nilai terkecil dari bagian array yang belum terurut, kemudian menukarnya dengan posisi saat ini. Proses pencarian nilai terkecil dilakukan berulang hingga semua data tersusun dengan benar. Algoritma ini membagi array menjadi bagian terurut dan belum terurut. Setiap langkah akan menambah satu elemen ke bagian yang sudah terurut.

Bubble Sort adalah algoritma sorting yang bekerja dengan membandingkan dua data yang bersebelahan lalu menukarnya jika urutannya salah. Proses ini dilakukan berulang sampai tidak ada lagi data yang perlu ditukar. Disebut Bubble Sort karena data terbesar akan “mengambang” ke akhir array seperti gelembung. Algoritma ini termasuk salah satu metode sorting paling sederhana.

2.2. Class InsertionSort_2511531011

```
package pekan7_2511531011;

public class InsertionSort_2511531011 {
    public static void insertionSort_2511531011(int[] arr_1011) {
        int n_1011 = arr_1011.length;
        for (int i = 1; i < n_1011; i++) {
            int key_1011 = arr_1011[i];
            int j_1011 = i - 1;
            while (j_1011 >= 0 && arr_1011[j_1011] > key_1011) {
                arr_1011[j_1011+1] = arr_1011[j_1011];
                j_1011--;
            }
            arr_1011[j_1011 + 1] = key_1011;
        }
    }

    public static void main(String[] args) {
        int arr_1011[] = {23, 78, 45, 8, 32, 56, 1};
        int n_1011 = arr_1011.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i = 0; i < n_1011; i++) {
            System.out.print(arr_1011[i] + " ");
        }
        System.out.println("");
        insertionSort_2511531011(arr_1011);
        System.out.printf("array yang terurut:\n");
        for (int i=0; i<n_1011; i++)
            System.out.print(arr_1011[i] + " ");
        System.out.println("");
    }
}
```

Kode program 2.1: InsertionSort_2511531011

Program ini digunakan untuk mengurutkan data angka menggunakan algoritma Insertion Sort. Pada method `insertionSort_2511531011()`, program akan mengambil satu angka sebagai key, lalu membandingkannya dengan angka-angka

sebelumnya untuk menentukan posisi yang tepat agar urutan menjadi dari kecil ke besar. Jika ada angka yang lebih besar dari key, maka angka tersebut digeser ke kanan sampai ditemukan posisi yang sesuai, kemudian key dimasukkan ke posisi tersebut. Pada method main, program terlebih dahulu menampilkan array yang belum terurut, kemudian memanggil proses insertion sort, dan akhirnya menampilkan array yang sudah terurut sehingga pengguna dapat melihat perubahan urutan data sebelum dan sesudah sorting dilakukan.

2.3. SelectionSort_2511531011

```
package pekan7_2511531011;

public class SelectionSort_2511531011 {
    public static void selectionSort_2511531011(int[] arr_1011) {
        int n_1011 = arr_1011.length;
        for (int i = 1; i < n_1011; i++) {
            int minIndex_1011 = i;
            for (int j_1011 = i + 1; j_1011 < n_1011; j_1011++)
            {
                if (arr_1011[j_1011] <
arr_1011[minIndex_1011]) {
                    minIndex_1011 = j_1011;
                }
            }
            int temp_1011 = arr_1011[i];
            arr_1011[i] = arr_1011[minIndex_1011];
            arr_1011[minIndex_1011] = temp_1011;
        }
    }

    public static void main(String[] args) {
        int arr_1011[] = {23, 78, 45, 8, 32, 56, 1};
        int n_1011 = arr_1011.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i = 0; i < n_1011; i++) {
            System.out.print(arr_1011[i] + " ");
        }
    }
}
```

```

    }
    System.out.println("");
    selectionSort_2511531011(arr_1011);
    System.out.printf("array yang terurut:\n");
    for (int i=0; i<n_1011; i++)
        System.out.print(arr_1011[i] + " ");
    System.out.println("");
}
}

```

Kode program 2.2: SelectionSort_2511531011

Program ini digunakan untuk mengurutkan data angka menggunakan algoritma Selection Sort. Pada method selectionSort_2511531011(), program akan mencari nilai terkecil dari bagian array yang belum terurut, kemudian menukarnya dengan data pada posisi saat ini agar urutan menjadi dari kecil ke besar. Proses pencarian nilai terkecil dilakukan berulang menggunakan perulangan bersarang sampai seluruh data tersusun dengan rapi. Pada method main, program menampilkan array sebelum diurutkan, lalu memanggil proses selection sort, dan akhirnya menampilkan array yang sudah terurut sehingga pengguna dapat melihat hasil perubahan data setelah sorting dilakukan.

2.4. BubbleSort_2511531011

```

package pekan7_2511531011;

public class BubbleSort_2511531011 {
    public static void bubbleSort_2511531011(int[] arr_1011) {
        int n_1011 = arr_1011.length;
        for (int i = 1; i<n_1011; i++) {
            for (int j_1011 = 0; j_1011 < n_1011 - 1; j_1011++)
            {
                if (arr_1011[j_1011] > arr_1011[j_1011 + 1])
                {
                    int temp_1011 = arr_1011[j_1011];
                    arr_1011[j_1011] = arr_1011[j_1011 +
1];

```



```

arr_1011[j_1011 + 1] = temp_1011;
    }
}
}

public static void main(String[] args) {
    int arr_1011[] = {23, 78, 45, 8, 32, 56, 1};
    int n_1011 = arr_1011.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i = 0; i < n_1011; i++) {
        System.out.print(arr_1011[i] + " ");
    }
    System.out.println("");
    bubbleSort_2511531011(arr_1011);
    System.out.printf("array yang terurut:\n");
    for (int i=0; i<n_1011; i++)
        System.out.print(arr_1011[i] + " ");
    System.out.println("");
}
}

```

Kode program 2.3: BubbleSort_2511531011

Program ini digunakan untuk mengurutkan data angka menggunakan algoritma Bubble Sort. Pada method bubbleSort_2511531011(), program akan membandingkan dua angka yang bersebelahan, lalu menukarnya jika urutannya salah sehingga angka yang lebih besar bergerak ke belakang array. Proses perbandingan dan pertukaran dilakukan secara berulang menggunakan perulangan bersarang sampai seluruh data tersusun dari kecil ke besar. Pada method main, program menampilkan array sebelum diurutkan, kemudian menjalankan proses bubble sort, dan akhirnya menampilkan array yang sudah terurut agar pengguna dapat melihat hasil perubahan urutan data setelah sorting dilakukan.

2.5. InsertionGUI_2511531011

```

package pekan7_2511531011;

```

```

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;
import java.lang.reflect.Array;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class InsertionGUI_2511531011 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_1011;
    private JLabel[] labelArray_1011;
    private JButton stepButton_1011, resetButton_1011,
setButton_1011;
    private JTextField inputField_1011;
    private JPanel panelArray_1011;
    private JTextArea stepArea_1011;

    private int i = 1, j;
    private boolean sorting_1011 = false;
    private int stepCount_1011 = 1;

    /**
     * Create the frame.

```

```

*/
public InsertionGUI_2511531011() {
    setTitle("Insertion Sort langkah per langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    //panel input
    JPanel inputPanel_1011 = new JPanel(new FlowLayout());
    inputField_1011 = new JTextField(30);
    setButton_1011 = new JButton("Set Array");
    inputPanel_1011.add(new JLabel ("Masukkan angka (pisahkan
dengan koma): "));
    inputPanel_1011.add(inputField_1011);
    inputPanel_1011.add(setButton_1011);

    //panel array visual
    panelArray_1011 = new JPanel();
    panelArray_1011.setLayout(new FlowLayout());

    //panel kontrol
    JPanel controlPanel_1011 = new JPanel();
    stepButton_1011 = new JButton("Langkah selanjutnya");
    resetButton_1011 = new JButton("Reset");
    stepButton_1011.setEnabled(false);
    controlPanel_1011.add(stepButton_1011);
    controlPanel_1011.add(resetButton_1011);

    //Area text untuk log langkah-langkah
    stepArea_1011 = new JTextArea(8,60);
    stepArea_1011.setEditable(false);
    stepArea_1011.setFont(new Font("Monospaced", Font.PLAIN,
14));

    JScrollPane scrollPane_1011 = new
JScrollPane(stepArea_1011);

    // tambahkan panel ke frame

```

```

        add(inputPanel_1011, BorderLayout.NORTH);
        add(panelArray_1011, BorderLayout.CENTER);
        add(controlPanel_1011, BorderLayout.SOUTH);
        add(scrollPane_1011, BorderLayout.EAST);

        // event set array
        setButton_1011.addActionListener(e ->
setArrayFromInput_1011());

        // event langkah selanjutnya
        stepButton_1011.addActionListener(e -> performStep_1011());

        // event reset
        resetButton_1011.addActionListener(e -> reset_1011());

    }

    private void setArrayFromInput_1011() {
        String text_1011 = inputField_1011.getText().trim();
        if (text_1011.isEmpty()) return;
        String[] parts_1011 = text_1011.split(", ");
        array_1011 = new int [parts_1011.length];
        try {
            for (int k = 0; k < parts_1011.length; k++) {
                array_1011[k] =
Integer.parseInt(parts_1011[k].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya
angka yang dipisahkan "
+ "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        i = 1;
        stepCount_1011 = 1;
        sorting_1011 = true;
        stepButton_1011.setEnabled(true);
    }

```

```

        stepArea_1011.setText("");
        panelArray_1011.removeAll();
        labelArray_1011 = new JLabel[array_1011.length];
        for (int k = 0; k < array_1011.length; k++) {
            labelArray_1011[k] = new
JLabel(String.valueOf(array_1011[k]));
            labelArray_1011[k].setFont(new Font("Arial",
Font.BOLD, 24));

            labelArray_1011[k].setBorder(BorderFactory.createLineBorder(Color
.BLACK));

            labelArray_1011[k].setPreferredSize(new
Dimension(50, 50));

            labelArray_1011[k].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_1011.add(labelArray_1011[k]);
        }
        panelArray_1011.revalidate();
        panelArray_1011.repaint();
    }

    private void performStep_1011() {
        if (i < array_1011.length && sorting_1011) {
            int key_1011 = array_1011[i];
            j = i - 1;

            StringBuilder stepLog_1011 = new StringBuilder();
            stepLog_1011.append("Langkah
").append(stepCount_1011).append(": Masukkan
").append(key_1011).append("\n");

            while (j >= 0 && array_1011[j] > key_1011) {
                array_1011[j + 1] = array_1011[j];
                j--;
            }
            array_1011[j + 1] = key_1011;

            updateLabels_1011();

```

```

        stepLog_1011.append("Hasil:
").append(arrayToString_1011(array_1011)).append("\n\n");
        stepArea_1011.append(stepLog_1011.toString());

        i++;
        stepCount_1011++;

        if (i == array_1011.length) {
            sorting_1011 = false;
            stepButton_1011.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting
selesai!");
        }
    }

    private void updateLabels_1011() {
        for (int k = 0; k < array_1011.length; k++) {

            labelArray_1011[k].setText((String.valueOf(array_1011[k])));
        }
    }

    private void reset_1011() {
        inputField_1011.setText("");
        panelArray_1011.removeAll();
        panelArray_1011.revalidate();
        panelArray_1011.repaint();
        stepArea_1011.setText("");
        stepButton_1011.setEnabled(false);
        sorting_1011 = false;
        i = 1;
        stepCount_1011 = 1;
    }

    private String arrayToString_1011(int[] arr_1011) {
        StringBuilder sb_1011 = new StringBuilder();
        for (int k = 0; k < arr_1011.length; k++) {

```

```

        sb_1011.append(arr_1011[k]);
        if (k < arr_1011.length - 1) sb_1011.append(", ");
    }
    return sb_1011.toString();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        InsertionGUI_2511531011 gui_1011 = new
InsertionGUI_2511531011();
        gui_1011.setVisible(true);
    });
}
}

```

Kode program 2.4: InsertionGUI_2511531011

Program ini merupakan aplikasi GUI berbasis Java Swing yang digunakan untuk memvisualisasikan proses algoritma Insertion Sort secara langkah demi langkah. Pengguna dapat memasukkan beberapa angka ke dalam program, kemudian program akan mengurutkan data tersebut dari kecil ke besar sambil menampilkan setiap proses perubahan array pada area log langkah. Selain menampilkan hasil sorting, program juga memperlihatkan visual array menggunakan label sehingga pengguna dapat memahami bagaimana algoritma Insertion Sort bekerja secara lebih jelas dan interaktif. Dengan adanya tombol langkah selanjutnya, proses pengurutan dapat diamati satu per satu sehingga cocok digunakan sebagai media pembelajaran algoritma sorting.

- **Method InsertionGUI_2511531011():** membuat tampilan GUI utama seperti panel input, tombol, area visual array, dan area log langkah sorting.
- **Method setArrayFromInput_1011():** membaca input angka dari pengguna lalu mengubahnya menjadi array integer yang akan diurutkan.
- **Method performStep_1011():** menjalankan satu langkah proses Insertion Sort setiap kali tombol langkah selanjutnya ditekan.

- **Method updateLabels_1011():** memperbarui tampilan angka pada label agar sesuai dengan isi array terbaru setelah sorting dilakukan.
- **Method reset_1011():** mengembalikan program ke kondisi awal dengan menghapus input, log langkah, dan tampilan array.
- **Method arrayToString_1011(int[] arr_1011):** mengubah isi array menjadi bentuk string agar dapat ditampilkan pada area log langkah sorting.
- **Method main:** titik awal program untuk menjalankan dan menampilkan GUI Insertion Sort.
- **Algoritma Insertion Sort:** mengambil satu data sebagai key lalu menyisipkannya ke posisi yang tepat pada bagian array yang sudah terurut.

2.6. Tugas Pekan 7

A. MahasiswaADT_2511531011

```
package pekan7_2511531011;

public class MahasiswaADT_2511531011 {
    String nama_1011;
    String nim_1011;
    String prodi_1011;

    public MahasiswaADT_2511531011 (String nama_1011, String
nim_1011, String prodi_1011) {
        this.nama_1011 = nama_1011;
        this.nim_1011 = nim_1011;
        this.prodi_1011 = prodi_1011;
    }

    //method getter
    public String getNama_1011() { return nama_1011; }
    public String getNim_1011() { return nim_1011; }
    public String getProdi_1011() { return prodi_1011; }
```



```

        //method setter
        public void setName_1011(String nama_1011) { this.nama_1011
= nama_1011; }
        public void setNim_1011(String nim_1011) { this.nim_1011 =
nim_1011; }
        public void setProdi_1011(String prodi_1011) {
this.prodi_1011 = prodi_1011; }

        //untuk cetak data mahasiswa
        @Override
        public String toString () {
            return "Nama: " + nama_1011 + ", NIM: " + nim_1011 +
", Prodi: " + prodi_1011;
        }

        //compare untuk mempermudah sorting
        public int compareTo(MahasiswaADT_2511531011 other) {
            return this.nim_1011.compareTo(other.nim_1011);
        }

        public static void main(String[] args) {

        }
    }
}

```

Kode Program 2.5: MahasiswaADT_2511531011

Pada kode program ini kita akan membuat konstruktor untuk algoritma sorting mahasiswa dengan atribut nama, nim, prodi. Disini kita juga membuat method setter dan getter untuk setiap atribut yang ada. Kita juga menggunakan override untuk mencetak data mahasiswa.

B. GUIutama_2511531011

```
package pekan7_2511531011;
```

```

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.*;

public class GUIutama_2511531011 extends JFrame {

    private static final long serialVersionUID = 1L;
    private MahasiswaADT_2511531011[] array_1011;
    private JLabel[] labelArray_1011;
    private JButton stepButton_1011, resetButton_1011,
setButton_1011;
    private JTextField inputField_1011;
    private JPanel panelArray_1011;
    private JTextArea stepArea_1011;
    private JComboBox<String> sortingPick_1011;

    private int i_1011 = 1, pass_1011 = 0;
    private boolean sorting_1011 = false;
    private int stepCount_1011 = 1;

    /**
     * Create the frame.
     */
    public GUIutama_2511531011() {
        setTitle("Sorting Mahasiswa");
        setSize(950, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        getContentPane().setLayout(new BorderLayout());

        //panel input
        JPanel inputPanel_1011 = new JPanel(new FlowLayout());
        inputField_1011 = new JTextField(40);

```

```

        setButton_1011 = new JButton("Set Array");
        inputPanel_1011.add(new JLabel ("Masukkan data (Nama-NIM-
Prodi): "));
        inputPanel_1011.add(inputField_1011);
        inputPanel_1011.add(setButton_1011);

        sortingPick_1011 = new JComboBox<>();
        sortingPick_1011.setModel(new DefaultComboBoxModel<>(new
String[] {
            "Insertion Sort", "Selection Sort", "Bubble
Sort"
        }));
        inputPanel_1011.add(new JLabel("Pilih Sorting: "));
        inputPanel_1011.add(sortingPick_1011);

        //panel array visual
        panelArray_1011 = new JPanel();
        panelArray_1011.setLayout(new FlowLayout());

        //panel kontrol
        JPanel controlPanel_1011 = new JPanel();
        stepButton_1011 = new JButton("Langkah selanjutnya");
        resetButton_1011 = new JButton("Reset");
        stepButton_1011.setEnabled(false);
        controlPanel_1011.add(stepButton_1011);
        controlPanel_1011.add(resetButton_1011);

        //Area text untuk log langkah-langkah
        stepArea_1011 = new JTextArea(30,60);
        stepArea_1011.setEditable(false);
        stepArea_1011.setFont(new Font("Monospaced", Font.PLAIN,
14));

        JScrollPane scrollPane_1011 = new
JScrollPane(stepArea_1011);
        getContentPane().add(controlPanel_1011,
BorderLayout.SOUTH);
        getContentPane().add(scrollPane_1011, BorderLayout.EAST);
        getContentPane().add(inputPanel_1011, BorderLayout.WEST);

```

```

        // tambahkan panel ke frame
        getContentPane().add(inputPanel_1011, BorderLayout.NORTH);
        getContentPane().add(panelArray_1011, BorderLayout.CENTER);

        // event set array
        setButton_1011.addActionListener(e ->
setArrayFromInput_1011());

        // event langkah selanjutnya
        stepButton_1011.addActionListener(e -> {
            String pilihan_1011 =
sortingPick_1011.getSelectedItem().toString();
            switch (pilihan_1011) {
                case "Insertion Sort":
                    performInsertionStep_1011();
                    break;
                case "Selection Sort":
                    performSelectionStep_1011();
                    break;
                case "Bubble Sort":
                    performBubbleStep_1011();
                    break;
            }
        });

        // event reset
        resetButton_1011.addActionListener(e -> reset_1011());

    }

    //set input
    private void setArrayFromInput_1011() {
        String text_1011 = inputField_1011.getText().trim();
        if (text_1011.isEmpty()) return;
        try {
            String[] data_1011 = text_1011.split(", ");

```

```

        array_1011 = new
MahasiswaADT_2511531011[data_1011.length];
        for (int k = 0; k < data_1011.length; k++) {
            String[] bagian_1011 = data_1011[k].split("-
");
            array_1011[k] = new
MahasiswaADT_2511531011(bagian_1011[0], bagian_1011[1], bagian_1011[2]);
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Format Salah!
\n"
                                + "Contoh: Aliifah-1011-IF", "Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }

    //reset variabel sorting
    i_1011 = 1;
    pass_1011 = 0;
    stepCount_1011 = 1;
    sorting_1011 = true;
    stepButton_1011.setEnabled(true);
    stepArea_1011.setText("");

    //tampilkan data
    panelArray_1011.removeAll();
    labelArray_1011 = new JLabel[array_1011.length];
    for (int k = 0; k < array_1011.length; k++) {
        labelArray_1011[k] = new
JLabel(array_1011[k].toString());
        labelArray_1011[k].setFont(new Font("Arial",
Font.BOLD, 14));

        labelArray_1011[k].setBorder(BorderFactory.createLineBorder(Color
.BLACK));

        labelArray_1011[k].setPreferredSize(new
Dimension(180, 50));

```

```

        labelArray_1011[k].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_1011.add(labelArray_1011[k]);
    }
    panelArray_1011.revalidate();
    panelArray_1011.repaint();
}

//insertion sort
private void performInsertionStep_1011() {
    if (i_1011 < array_1011.length && sorting_1011) {
        MahasiswaADT_2511531011 key_1011 =
array_1011[i_1011];
        int j = i_1011 - 1;

        StringBuilder stepLog_1011 = new StringBuilder();
        stepLog_1011.append("Langkah
").append(stepCount_1011).append(": Masukkan
").append(key_1011).append("\n");

        while (j >= 0 && array_1011[j].compareTo(key_1011) >
0) {
            array_1011[j + 1] = array_1011[j];
            j--;
        }
        array_1011[j + 1] = key_1011;

        logStep_1011("InsertionSort");
        updateLabels_1011();
        i_1011++;
        checkFinished_1011(i_1011>=array_1011.length);
    }
}

private void performSelectionStep_1011() {
    if (pass_1011 < array_1011.length - 1 && sorting_1011) {
        int min_1011 = pass_1011;
        for (int k = pass_1011 + 1; k < array_1011.length; k++) {

```

```

        if (array_1011[k].compareTo(array_1011[min_1011]) <
0) {

            min_1011 = k;

        }

    }

    MahasiswaADT_2511531011 temp_1011 = array_1011[pass_1011];
    array_1011[pass_1011] = array_1011[min_1011];
    array_1011[min_1011] = temp_1011;

    logStep_1011("Selection Sort");
    updateLabels_1011();
    pass_1011++;
    checkFinished_1011(pass_1011 >= array_1011.length - 1);
}

private void performBubbleStep_1011() {
    if (pass_1011 < array_1011.length - 1 && sorting_1011) {
        for (int k = 0; k < array_1011.length - 1 - pass_1011; k++)
        {

            if (array_1011[k].compareTo(array_1011[k +
1]) > 0) {

                MahasiswaADT_2511531011 temp_1011 =
array_1011[k];

                array_1011[k] = array_1011[k + 1];
                array_1011[k + 1] = temp_1011;

            }

        }

        logStep_1011("Bubble Sort");
        updateLabels_1011();
        pass_1011++;
        checkFinished_1011(pass_1011 >= array_1011.length - 1);
    }

}

//update label
private void updateLabels_1011() {
    for (int k = 0; k < array_1011.length; k++) {

```

```

        labelArray_1011[k].setText((array_1011[k].toString()));
    }
}

//log step
private void logStep_1011(String metode_1011) {
    stepArea_1011.append("Langkah " + stepCount_1011 + " : ");
    stepArea_1011.append(arrayToString_1011() + "\n");
    stepCount_1011++;
}

//cek finish
private void checkFinished_1011(boolean selesai_1011) {
    if (selesai_1011) {
        sorting_1011 = false;
        stepButton_1011.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting
Selesai!");
    }
}

private void reset_1011() {
    inputField_1011.setText("");
    panelArray_1011.removeAll();
    panelArray_1011.revalidate();
    panelArray_1011.repaint();
    stepArea_1011.setText("");
    stepButton_1011.setEnabled(false);
    sorting_1011 = false;
    i_1011 = 1;
    pass_1011 = 0;
    stepCount_1011 = 1;
}

private String arrayToString_1011() {
    StringBuilder sb_1011 = new StringBuilder();
    sb_1011.append("[");

```



```

        for (int k = 0; k < array_1011.length; k++) {
            sb_1011.append(array_1011[k].toString());
            if (k < array_1011.length - 1) sb_1011.append(", ");
        }
        sb_1011.append("]");
        return sb_1011.toString();
    }

    public static void main (String[] args) {
        new GUIutama_2511531011().setVisible(true);
    }
}

```

Kode program 2.6: GUIutama_2511531011

Pada program ini kita akan membuat sebuah GUI untuk melakukan proses sorting. Kita akan menggunakan JFrame dan melakukan coding manual untuk membuat berbagai macam field, textbox, combobox, dan lainnya. Disini kita menggunakan banyak sekali method, diantaranya:

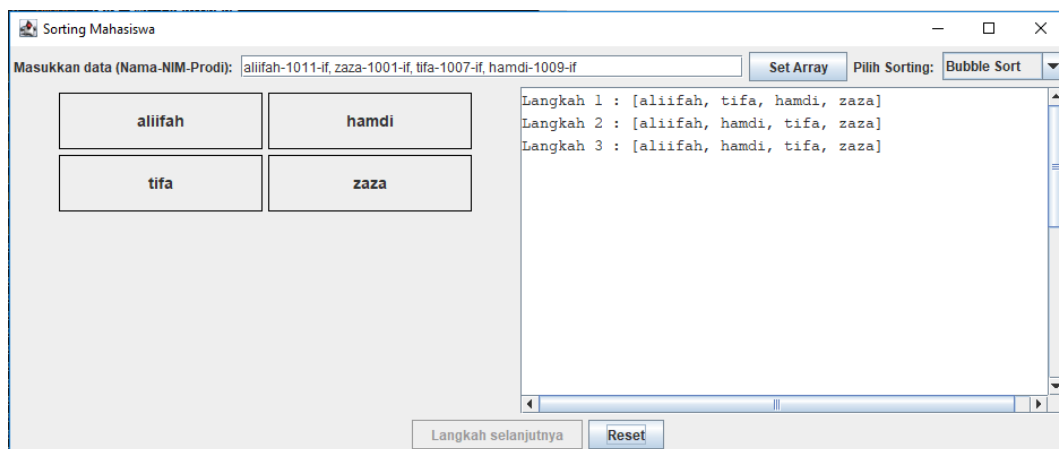
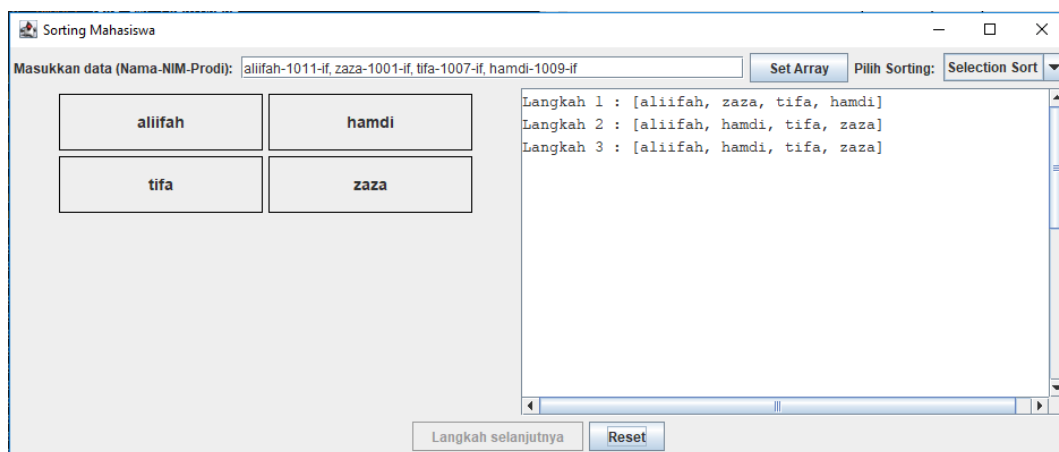
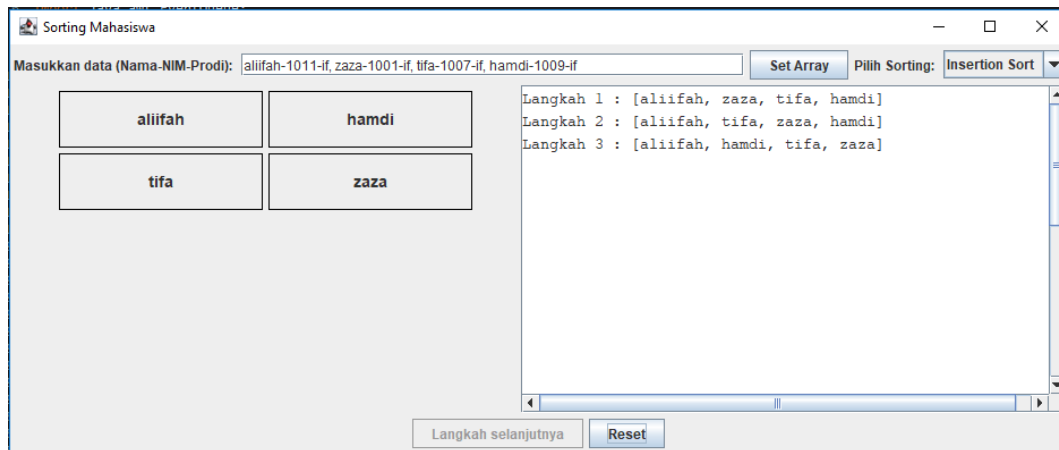
1. Method `setArrayFromInput_1011()`
 - Method ini digunakan untuk membaca input data mahasiswa dari text field.
 - Data yang dimasukkan dipisahkan menggunakan tanda koma dan tanda strip untuk membentuk objek mahasiswa.
 - Setelah data berhasil dimasukkan, method akan mereset variabel sorting dan menampilkan data ke panel GUI.
 - Method ini juga menangani kesalahan input agar program tidak error ketika format salah.
2. Method `performInsertionStep_1011()`
 - Method ini menjalankan algoritma Insertion Sort secara bertahap setiap kali tombol ditekan.
 - Data mahasiswa dibandingkan berdasarkan nama menggunakan method `compareTo()`.

- Jika ada data yang lebih besar dari key, maka data tersebut digeser ke kanan.
 - Setelah proses selesai, tampilan label dan langkah sorting akan diperbarui pada GUI.
3. Method performSelectionStep_1011()
- Method ini menjalankan algoritma Selection Sort langkah demi langkah.
 - Program mencari data dengan nilai terkecil dari bagian array yang belum terurut.
 - Setelah ditemukan, data tersebut ditukar dengan posisi awal pada pass saat itu.
 - Hasil sorting kemudian ditampilkan kembali ke dalam GUI dan area log langkah.
4. Method performBubbleStep_1011()
- Method ini digunakan untuk menjalankan algoritma Bubble Sort secara bertahap.
 - Program membandingkan dua data yang bersebelahan lalu menukarnya jika urutannya salah.
 - Proses dilakukan berulang sampai data terbesar berada di posisi akhir array.
 - Setelah satu pass selesai, tampilan data dan langkah sorting akan diperbarui.
5. Method updateLabels_1011()
- Method ini berfungsi untuk memperbarui isi label pada panel array.
 - Setiap label akan menampilkan data mahasiswa terbaru setelah proses sorting dilakukan.
 - Method ini membuat tampilan GUI selalu sesuai dengan isi array sebenarnya.
 - Dengan demikian, pengguna dapat melihat perubahan urutan data secara langsung.
6. Method logStep_1011(String metode_1011)
- Method ini digunakan untuk mencatat setiap langkah sorting ke dalam JTextArea.

- Program akan menampilkan nomor langkah dan isi array setelah proses sorting berjalan.
 - Method ini membantu pengguna memahami proses perubahan urutan data.
 - Setiap langkah baru akan ditambahkan ke bawah log sebelumnya.
7. Method `checkFinished_1011(boolean selesai_1011)`
- Method ini digunakan untuk memeriksa apakah proses sorting sudah selesai atau belum.
 - Jika sorting selesai, tombol langkah berikutnya akan dinonaktifkan.
 - Program juga menampilkan pesan bahwa proses sorting telah selesai dilakukan.
 - Method ini mencegah pengguna menekan tombol sorting setelah data sudah terurut.
8. Method `reset_1011()`
- Method ini digunakan untuk mengembalikan program ke kondisi awal.
 - Semua input, panel array, dan area log langkah akan dikosongkan kembali.
 - Variabel sorting juga direset agar program bisa digunakan ulang dari awal.
 - Tombol langkah berikutnya akan dinonaktifkan sampai data baru dimasukkan.
9. Method `arrayToString_1011`
- Method ini digunakan untuk mengubah isi array menjadi bentuk teks.
 - Data mahasiswa akan digabung menjadi satu string dengan format array.
 - Hasil string tersebut digunakan untuk ditampilkan pada area log langkah sorting.
 - Method ini mempermudah pengguna melihat isi array setelah setiap proses sorting.
10. Method `main`
- Method `main` merupakan titik awal program dijalankan.
 - Program akan membuat objek `GUIutama_2511531011` dan menampilkan frame GUI.

- Method ini memastikan aplikasi dapat berjalan ketika dieksekusi.
- Tanpa method ini, program Java tidak dapat dimulai.

Output:



BAB III

KESIMPULAN

3.1. Kesimpulan

Penggunaan algoritma sorting lebih memudahkan kita dalam mengurutkan sebuah data dari list. Algoritma sorting memiliki banyak variasi, diantaranya: selection sort, insertion sort, dan bubble sort. Algoritma sorting ini juga memiliki kelebihan dan kekurangannya masing-masing jika diaplikasikan dalam sebuah program. Oleh karena itu, dalam praktikum kali ini kita mengetahui tentang macam-macam algoritma sorting dan bagaimana penerapannya dalam pemrograman.

3.2. Saran

Karena banyaknya jenis dari algoritma sorting, mahasiswa hendaknya terus memahami dan mempelajari bagaimana pennggunaan dari algoritma sorting. Tidak hanya melalui praktikum, mahasiswa juga dapat melakukan latihan melalui tugas maupun proyek pribadi dalam penggunaan algoritma sorting. Dari sini diharapkan juga mahasiswa mampu memahami sebagian algoritma sorting dan penerapannya pada program dan GUI.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, 3rd ed., Cambridge, MA: MIT Press, 2009.
- [2] Wahyudi, *Struktur Data dan Algoritma*, Padang: Informatika, 2026